

Booth Multiplier Structural

Prompt Strategy: Zero Shot

Prompt:

Generate synthesizable Verilog code for a 16-bit signed Booth Multiplier.

Requirements:

- Implement Radix-2 Booth multiplication.
- Use Structural RTL style only.
- Target FPGA: Xilinx Artix-7 xc7a100tcsg324-2.
- Use explicit module hierarchy and module instantiations.
- Use Booth encoding and partial-product generation.
- Do not use behavioral constructs.
- Do not use always, always_comb, or always_ff blocks.
- Do not use the multiplication operator (*).
- Inputs:
 - input signed [15:0] multiplicand
 - input signed [15:0] multiplier
- Output:
 - output signed [31:0] product
- Generate synthesizable Verilog only.
- Output code only.

Code:

```
module booth_encoder (
    input wire y_i,
    input wire y_im1,
    output wire add,
    output wire sub
);
    assign add = y_i & ~y_im1;
    assign sub = ~y_i & y_im1;
endmodule

module pp_gen (
    input wire signed [15:0] multiplicand,
    input wire add,
    input wire sub,
    input wire [4:0] shift,
    output wire signed [31:0] pp
);
    wire signed [31:0] m_ext;
    wire signed [31:0] pos_pp;
    wire signed [31:0] neg_pp;

    assign m_ext = {{16{multiplicand[15]}}, multiplicand};
    assign pos_pp = m_ext <<< shift;
    assign neg_pp = (~pos_pp) + 32'sd1;

    assign pp = add ? pos_pp :
                sub ? neg_pp :
                32'sd0;
endmodule

module adder32 (
    input wire [31:0] a,
    input wire [31:0] b,
    output wire [31:0] s
);
    wire [32:0] c;

    assign c[0] = 1'b0;

    genvar i;
    generate
        for (i = 0; i < 32; i = i + 1) begin : GEN_ADD
            assign s[i] = a[i] ^ b[i] ^ c[i];
            assign c[i+1] = (a[i] & b[i]) |
                            (a[i] & c[i]) |
                            (b[i] & c[i]);
        end
    endgenerate
endmodule
```

```

        (b[i] & c[i]);
    end
endgenerate
endmodule

```

```

module booth_multiplier_16 (
    input  wire signed [15:0] multiplicand,
    input  wire signed [15:0] multiplier,
    output wire signed [31:0] product
);
    wire add0, sub0;
    wire add1, sub1;
    wire add2, sub2;
    wire add3, sub3;
    wire add4, sub4;
    wire add5, sub5;
    wire add6, sub6;
    wire add7, sub7;
    wire add8, sub8;
    wire add9, sub9;
    wire add10, sub10;
    wire add11, sub11;
    wire add12, sub12;
    wire add13, sub13;
    wire add14, sub14;
    wire add15, sub15;

    booth_encoder BE0 (.y_i(multiplier[0]), .y_im1(1'b0), .add(add0), .sub(sub0));
    booth_encoder BE1 (.y_i(multiplier[1]), .y_im1(multiplier[0]), .add(add1), .sub(sub1));
    booth_encoder BE2 (.y_i(multiplier[2]), .y_im1(multiplier[1]), .add(add2), .sub(sub2));
    booth_encoder BE3 (.y_i(multiplier[3]), .y_im1(multiplier[2]), .add(add3), .sub(sub3));
    booth_encoder BE4 (.y_i(multiplier[4]), .y_im1(multiplier[3]), .add(add4), .sub(sub4));
    booth_encoder BE5 (.y_i(multiplier[5]), .y_im1(multiplier[4]), .add(add5), .sub(sub5));
    booth_encoder BE6 (.y_i(multiplier[6]), .y_im1(multiplier[5]), .add(add6), .sub(sub6));
    booth_encoder BE7 (.y_i(multiplier[7]), .y_im1(multiplier[6]), .add(add7), .sub(sub7));
    booth_encoder BE8 (.y_i(multiplier[8]), .y_im1(multiplier[7]), .add(add8), .sub(sub8));
    booth_encoder BE9 (.y_i(multiplier[9]), .y_im1(multiplier[8]), .add(add9), .sub(sub9));
    booth_encoder BE10 (.y_i(multiplier[10]), .y_im1(multiplier[9]), .add(add10), .sub(sub10));
    booth_encoder BE11 (.y_i(multiplier[11]), .y_im1(multiplier[10]), .add(add11), .sub(sub11));
    booth_encoder BE12 (.y_i(multiplier[12]), .y_im1(multiplier[11]), .add(add12), .sub(sub12));
    booth_encoder BE13 (.y_i(multiplier[13]), .y_im1(multiplier[12]), .add(add13), .sub(sub13));
    booth_encoder BE14 (.y_i(multiplier[14]), .y_im1(multiplier[13]), .add(add14), .sub(sub14));
    booth_encoder BE15 (.y_i(multiplier[15]), .y_im1(multiplier[14]), .add(add15), .sub(sub15));

    wire signed [31:0] pp0, pp1, pp2, pp3;
    wire signed [31:0] pp4, pp5, pp6, pp7;
    wire signed [31:0] pp8, pp9, pp10, pp11;
    wire signed [31:0] pp12, pp13, pp14, pp15;

```

```

    pp_gen PPG0 (.multiplicand(multiplicand), .add(add0), .sub(sub0), .shift(5'd0),
.pp(pp0));
    pp_gen PPG1 (.multiplicand(multiplicand), .add(add1), .sub(sub1), .shift(5'd1),
.pp(pp1));
    pp_gen PPG2 (.multiplicand(multiplicand), .add(add2), .sub(sub2), .shift(5'd2),
.pp(pp2));
    pp_gen PPG3 (.multiplicand(multiplicand), .add(add3), .sub(sub3), .shift(5'd3),
.pp(pp3));
    pp_gen PPG4 (.multiplicand(multiplicand), .add(add4), .sub(sub4), .shift(5'd4),
.pp(pp4));
    pp_gen PPG5 (.multiplicand(multiplicand), .add(add5), .sub(sub5), .shift(5'd5),
.pp(pp5));
    pp_gen PPG6 (.multiplicand(multiplicand), .add(add6), .sub(sub6), .shift(5'd6),
.pp(pp6));
    pp_gen PPG7 (.multiplicand(multiplicand), .add(add7), .sub(sub7), .shift(5'd7),
.pp(pp7));
    pp_gen PPG8 (.multiplicand(multiplicand), .add(add8), .sub(sub8), .shift(5'd8),
.pp(pp8));
    pp_gen PPG9 (.multiplicand(multiplicand), .add(add9), .sub(sub9), .shift(5'd9),
.pp(pp9));
    pp_gen PPG10 (.multiplicand(multiplicand), .add(add10), .sub(sub10), .shift(5'd10),
.pp(pp10));
    pp_gen PPG11 (.multiplicand(multiplicand), .add(add11), .sub(sub11), .shift(5'd11),
.pp(pp11));
    pp_gen PPG12 (.multiplicand(multiplicand), .add(add12), .sub(sub12), .shift(5'd12),
.pp(pp12));
    pp_gen PPG13 (.multiplicand(multiplicand), .add(add13), .sub(sub13), .shift(5'd13),
.pp(pp13));
    pp_gen PPG14 (.multiplicand(multiplicand), .add(add14), .sub(sub14), .shift(5'd14),
.pp(pp14));
    pp_gen PPG15 (.multiplicand(multiplicand), .add(add15), .sub(sub15), .shift(5'd15),
.pp(pp15));

```

```

wire [31:0] s0, s1, s2, s3, s4, s5, s6, s7;
wire [31:0] t0, t1, t2, t3;
wire [31:0] u0, u1;

```

```

adder32 A0 (.a(pp0), .b(pp1), .s(s0));
adder32 A1 (.a(pp2), .b(pp3), .s(s1));
adder32 A2 (.a(pp4), .b(pp5), .s(s2));
adder32 A3 (.a(pp6), .b(pp7), .s(s3));
adder32 A4 (.a(pp8), .b(pp9), .s(s4));
adder32 A5 (.a(pp10), .b(pp11), .s(s5));
adder32 A6 (.a(pp12), .b(pp13), .s(s6));
adder32 A7 (.a(pp14), .b(pp15), .s(s7));

```

```

adder32 B0 (.a(s0), .b(s1), .s(t0));

```

```

    adder32 B1 (.a(s2), .b(s3), .s(t1));
    adder32 B2 (.a(s4), .b(s5), .s(t2));
    adder32 B3 (.a(s6), .b(s7), .s(t3));

    adder32 C0 (.a(t0), .b(t1), .s(u0));
    adder32 C1 (.a(t2), .b(t3), .s(u1));

    adder32 D0 (.a(u0), .b(u1), .s(product));

endmodule

```

Prompt Strategy: Few Shot

Prompt:

Example of Structural RTL coding style:

```

module full_adder(
    input a,
    input b,
    input cin,
    output sum,
    output cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b & cin);
endmodule

```

Using the same Structural RTL style shown above, generate synthesizable Verilog code for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth multiplication.
- Structural RTL only.
- Explicit module hierarchy.
- Booth encoder modules and partial-product modules should be instantiated structurally.
- No behavioral blocks.
- No multiplication operator (*).
- Target FPGA: xc7a100tcsg324-2.
- Output code only.

Code:

```

module full_adder(
    input a,
    input b,
    input cin,
    output sum,
    output cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b & cin);
endmodule

```

```

module ripple_adder32(
    input [31:0] a,
    input [31:0] b,
    output [31:0] sum

```

```

);

wire [32:0] c;

assign c[0] = 1'b0;

genvar i;
generate
    for(i=0;i<32;i=i+1) begin : FA_GEN
        full_adder fa(
            .a(a[i]),
            .b(b[i]),
            .cin(c[i]),
            .sum(sum[i]),
            .cout(c[i+1])
        );
    end
endgenerate
endmodule

```

```

module ripple_adder32_cin1(
    input [31:0] a,
    input [31:0] b,
    output [31:0] sum
);
wire [32:0] c;

```

```

assign c[0] = 1'b1;

genvar i;

generate

    for(i=0;i<32;i=i+1) begin : FA_GEN

        full_adder fa(

            .a(a[i]),

            .b(b[i]),

            .cin(c[i]),

            .sum(sum[i]),

            .cout(c[i+1])

        );

    end

endgenerate

endmodule

```

```

module booth_encoder(

    input y_i,

    input y_im1,

    output add,

    output sub

);

assign add = y_i & ~y_im1; // 01

assign sub = ~y_i & y_im1; // 10

endmodule

```



```

module booth_partial_product #
(
    parameter SHIFT = 0
)
(
    input  [15:0] x,
    input      add,
    input      sub,
    output [31:0] pp
);
wire [31:0] x_ext;
wire [31:0] x_shift;
wire [31:0] x_shift_inv;
wire [31:0] x_shift_neg;

assign x_ext    = {{16{x[15]}}, x};
assign x_shift  = x_ext << SHIFT;
assign x_shift_inv = ~x_shift;

ripple_adder32_cin1 neg_gen(
    .a(x_shift_inv),
    .b(32'b0),
    .sum(x_shift_neg)
);

genvar k;

```

```

generate
    for(k=0;k<32;k=k+1) begin : PP_SEL
        assign pp[k] =
            (add & x_shift[k]) |
            (sub & x_shift_neg[k]);
    end
endgenerate
endmodule

```

```

module booth_multiplier_16x16_signed(
    input [15:0] multiplicand,
    input [15:0] multiplier,
    output [31:0] product
);

```

```

    wire add0, sub0;
    wire add1, sub1;
    wire add2, sub2;
    wire add3, sub3;
    wire add4, sub4;
    wire add5, sub5;
    wire add6, sub6;
    wire add7, sub7;
    wire add8, sub8;
    wire add9, sub9;
    wire add10, sub10;

```

wire add11, sub11;

wire add12, sub12;

wire add13, sub13;

wire add14, sub14;

wire add15, sub15;

booth_encoder be0 (.y_i(multiplier[0]), .y_im1(1'b0), .add(add0), .sub(sub0));

booth_encoder be1 (.y_i(multiplier[1]), .y_im1(multiplier[0]), .add(add1), .sub(sub1));

booth_encoder be2 (.y_i(multiplier[2]), .y_im1(multiplier[1]), .add(add2), .sub(sub2));

booth_encoder be3 (.y_i(multiplier[3]), .y_im1(multiplier[2]), .add(add3), .sub(sub3));

booth_encoder be4 (.y_i(multiplier[4]), .y_im1(multiplier[3]), .add(add4), .sub(sub4));

booth_encoder be5 (.y_i(multiplier[5]), .y_im1(multiplier[4]), .add(add5), .sub(sub5));

booth_encoder be6 (.y_i(multiplier[6]), .y_im1(multiplier[5]), .add(add6), .sub(sub6));

booth_encoder be7 (.y_i(multiplier[7]), .y_im1(multiplier[6]), .add(add7), .sub(sub7));

booth_encoder be8 (.y_i(multiplier[8]), .y_im1(multiplier[7]), .add(add8), .sub(sub8));

booth_encoder be9 (.y_i(multiplier[9]), .y_im1(multiplier[8]), .add(add9), .sub(sub9));

booth_encoder be10 (.y_i(multiplier[10]), .y_im1(multiplier[9]), .add(add10), .sub(sub10));

booth_encoder be11 (.y_i(multiplier[11]), .y_im1(multiplier[10]), .add(add11), .sub(sub11));

booth_encoder be12 (.y_i(multiplier[12]), .y_im1(multiplier[11]), .add(add12), .sub(sub12));

booth_encoder be13 (.y_i(multiplier[13]), .y_im1(multiplier[12]), .add(add13), .sub(sub13));

booth_encoder be14 (.y_i(multiplier[14]), .y_im1(multiplier[13]), .add(add14), .sub(sub14));

booth_encoder be15 (.y_i(multiplier[15]), .y_im1(multiplier[14]), .add(add15), .sub(sub15));

wire [31:0] pp0;

wire [31:0] pp1;

wire [31:0] pp2;

wire [31:0] pp3;

wire [31:0] pp4;

wire [31:0] pp5;

wire [31:0] pp6;

wire [31:0] pp7;

wire [31:0] pp8;

wire [31:0] pp9;

wire [31:0] pp10;

wire [31:0] pp11;

wire [31:0] pp12;

wire [31:0] pp13;

wire [31:0] pp14;

wire [31:0] pp15;

booth_partial_product #(.SHIFT(0)) ppg0 (.x(multiplicand), .add(add0), .sub(sub0),
.pp(pp0));

booth_partial_product #(.SHIFT(1)) ppg1 (.x(multiplicand), .add(add1), .sub(sub1),
.pp(pp1));

booth_partial_product #(.SHIFT(2)) ppg2 (.x(multiplicand), .add(add2), .sub(sub2),
.pp(pp2));

booth_partial_product #(.SHIFT(3)) ppg3 (.x(multiplicand), .add(add3), .sub(sub3),
.pp(pp3));

booth_partial_product #(.SHIFT(4)) ppg4 (.x(multiplicand), .add(add4), .sub(sub4),
.pp(pp4));

booth_partial_product #(.SHIFT(5)) ppg5 (.x(multiplicand), .add(add5), .sub(sub5),
.pp(pp5));

booth_partial_product #(.SHIFT(6)) ppg6 (.x(multiplicand), .add(add6), .sub(sub6),
.pp(pp6));

booth_partial_product #(.SHIFT(7)) ppg7 (.x(multiplicand), .add(add7), .sub(sub7),
.pp(pp7));

```
booth_partial_product #(.SHIFT(8 )) ppg8 (.x(multiplicand), .add(add8), .sub(sub8),  
.pp(pp8));  
  
booth_partial_product #(.SHIFT(9 )) ppg9 (.x(multiplicand), .add(add9), .sub(sub9),  
.pp(pp9));  
  
booth_partial_product #(.SHIFT(10)) ppg10 (.x(multiplicand), .add(add10), .sub(sub10),  
.pp(pp10));  
  
booth_partial_product #(.SHIFT(11)) ppg11 (.x(multiplicand), .add(add11), .sub(sub11),  
.pp(pp11));  
  
booth_partial_product #(.SHIFT(12)) ppg12 (.x(multiplicand), .add(add12), .sub(sub12),  
.pp(pp12));  
  
booth_partial_product #(.SHIFT(13)) ppg13 (.x(multiplicand), .add(add13), .sub(sub13),  
.pp(pp13));  
  
booth_partial_product #(.SHIFT(14)) ppg14 (.x(multiplicand), .add(add14), .sub(sub14),  
.pp(pp14));  
  
booth_partial_product #(.SHIFT(15)) ppg15 (.x(multiplicand), .add(add15), .sub(sub15),  
.pp(pp15));
```

```
wire [31:0] s1;
```

```
wire [31:0] s2;
```

```
wire [31:0] s3;
```

```
wire [31:0] s4;
```

```
wire [31:0] s5;
```

```
wire [31:0] s6;
```

```
wire [31:0] s7;
```

```
wire [31:0] s8;
```

```
wire [31:0] s9;
```

```
wire [31:0] s10;
```

```
wire [31:0] s11;
```

```
wire [31:0] s12;
```

```
wire [31:0] s13;
```

```
wire [31:0] s14;
```

```
wire [31:0] s15;
```

```
ripple_adder32 a1 (.a(pp0), .b(pp1), .sum(s1));
```

```
ripple_adder32 a2 (.a(s1), .b(pp2), .sum(s2));
```

```
ripple_adder32 a3 (.a(s2), .b(pp3), .sum(s3));
```

```
ripple_adder32 a4 (.a(s3), .b(pp4), .sum(s4));
```

```
ripple_adder32 a5 (.a(s4), .b(pp5), .sum(s5));
```

```
ripple_adder32 a6 (.a(s5), .b(pp6), .sum(s6));
```

```
ripple_adder32 a7 (.a(s6), .b(pp7), .sum(s7));
```

```
ripple_adder32 a8 (.a(s7), .b(pp8), .sum(s8));
```

```
ripple_adder32 a9 (.a(s8), .b(pp9), .sum(s9));
```

```
ripple_adder32 a10 (.a(s9), .b(pp10), .sum(s10));
```

```
ripple_adder32 a11 (.a(s10), .b(pp11), .sum(s11));
```

```
ripple_adder32 a12 (.a(s11), .b(pp12), .sum(s12));
```

```
ripple_adder32 a13 (.a(s12), .b(pp13), .sum(s13));
```

```
ripple_adder32 a14 (.a(s13), .b(pp14), .sum(s14));
```

```
ripple_adder32 a15 (.a(s14), .b(pp15), .sum(s15));
```

```
assign product = s15;
```

```
endmodule
```

Prompt Strategy: Chain of Thought

Prompt:

Design a 16-bit signed Booth Multiplier.

Before generating the RTL:

1. Analyze the Booth multiplication algorithm.
2. Determine the Booth encoding logic.
3. Determine the partial-product generation structure.
4. Determine the adder hierarchy required.
5. Plan the structural datapath.
6. Plan the module hierarchy.

After completing the reasoning process, generate synthesizable Verilog.

Requirements:

- Radix-2 Booth multiplication.
- Structural RTL only.
- Explicit module instantiations.
- No behavioral blocks.
- No multiplication operator (*).
- Target FPGA: xc7a100tcsg324-2.
- Output complete code only.

Code:

```
module full_adder(  
    input a,
```

```

    input b,

    input cin,

    output sum,

    output cout

);

    assign sum = a ^ b ^ cin;

    assign cout = (a & b) | (a & cin) | (b & cin);

endmodule

```

```

module ripple_adder_32(

    input [31:0] a,

    input [31:0] b,

    output [31:0] sum

);

    wire [32:0] c;

    assign c[0] = 1'b0;

    genvar i;

    generate

        for(i=0;i<32;i=i+1) begin : FA_CHAIN

            full_adder fa (

                .a (a[i]),

                .b (b[i]),

                .cin (c[i]),

                .sum (sum[i]),

```



```

        .cout(c[i+1])

    );

end

endgenerate

endmodule

```

```

module booth_pp_gen(

    input  [15:0] a,

    input    y_i,

    input    y_im1,

    input  [4:0] shift,

    output [31:0] pp

);

    wire add_op;

    wire sub_op;


    wire [15:0] a_ext;

    wire [15:0] a_inv;

    wire [16:0] mag17;

    wire [31:0] mag32;

    wire [31:0] shifted_mag;

    wire [31:0] shifted_sub;


    assign add_op = y_i & ~y_im1;

    assign sub_op = ~y_i & y_im1;

```

```

assign a_ext = a;

assign a_inv = ~a_ext;

assign mag17 = sub_op ? {a_inv[15], a_inv} + 17'd1 :
                    {a_ext[15], a_ext};

assign mag32 = {{15{mag17[16]}}, mag17};

assign shifted_mag = mag32 << shift;

assign shifted_sub = (~shifted_mag) + 32'd1;

assign pp = add_op ? shifted_mag :
            sub_op ? shifted_sub :
            32'd0;
endmodule

```

```

module booth_multiplier_16s(
    input [15:0] multiplicand,
    input [15:0] multiplier,
    output [31:0] product
);

```

```

    wire [31:0] pp0;
    wire [31:0] pp1;
    wire [31:0] pp2;
    wire [31:0] pp3;

```

```
wire [31:0] pp4;  
wire [31:0] pp5;  
wire [31:0] pp6;  
wire [31:0] pp7;  
wire [31:0] pp8;  
wire [31:0] pp9;  
wire [31:0] pp10;  
wire [31:0] pp11;  
wire [31:0] pp12;  
wire [31:0] pp13;  
wire [31:0] pp14;  
wire [31:0] pp15;
```

```
    booth_pp_gen g0 (.a(multiplicand), .y_i(multiplier[0]), .y_im1(1'b0),      .shift(5'd0),  
    .pp(pp0));  
  
    booth_pp_gen g1 (.a(multiplicand), .y_i(multiplier[1]), .y_im1(multiplier[0]), .shift(5'd1),  
    .pp(pp1));  
  
    booth_pp_gen g2 (.a(multiplicand), .y_i(multiplier[2]), .y_im1(multiplier[1]), .shift(5'd2),  
    .pp(pp2));  
  
    booth_pp_gen g3 (.a(multiplicand), .y_i(multiplier[3]), .y_im1(multiplier[2]), .shift(5'd3),  
    .pp(pp3));  
  
    booth_pp_gen g4 (.a(multiplicand), .y_i(multiplier[4]), .y_im1(multiplier[3]), .shift(5'd4),  
    .pp(pp4));  
  
    booth_pp_gen g5 (.a(multiplicand), .y_i(multiplier[5]), .y_im1(multiplier[4]), .shift(5'd5),  
    .pp(pp5));  
  
    booth_pp_gen g6 (.a(multiplicand), .y_i(multiplier[6]), .y_im1(multiplier[5]), .shift(5'd6),  
    .pp(pp6));  
  
    booth_pp_gen g7 (.a(multiplicand), .y_i(multiplier[7]), .y_im1(multiplier[6]), .shift(5'd7),  
    .pp(pp7));
```

```

    booth_pp_gen g8 (.a(multiplicand), .y_i(multiplier[8]), .y_im1(multiplier[7]), .shift(5'd8),
.pp(pp8));

    booth_pp_gen g9 (.a(multiplicand), .y_i(multiplier[9]), .y_im1(multiplier[8]), .shift(5'd9),
.pp(pp9));

    booth_pp_gen g10 (.a(multiplicand), .y_i(multiplier[10]), .y_im1(multiplier[9]), .shift(5'd10),
.pp(pp10));

    booth_pp_gen g11 (.a(multiplicand), .y_i(multiplier[11]), .y_im1(multiplier[10]),.shift(5'd11),
.pp(pp11));

    booth_pp_gen g12 (.a(multiplicand), .y_i(multiplier[12]), .y_im1(multiplier[11]),.shift(5'd12),
.pp(pp12));

    booth_pp_gen g13 (.a(multiplicand), .y_i(multiplier[13]), .y_im1(multiplier[12]),.shift(5'd13),
.pp(pp13));

    booth_pp_gen g14 (.a(multiplicand), .y_i(multiplier[14]), .y_im1(multiplier[13]),.shift(5'd14),
.pp(pp14));

    booth_pp_gen g15 (.a(multiplicand), .y_i(multiplier[15]), .y_im1(multiplier[14]),.shift(5'd15),
.pp(pp15));

```

```

wire [31:0] s1;

wire [31:0] s2;

wire [31:0] s3;

wire [31:0] s4;

wire [31:0] s5;

wire [31:0] s6;

wire [31:0] s7;

wire [31:0] s8;

wire [31:0] s9;

wire [31:0] s10;

wire [31:0] s11;

wire [31:0] s12;

wire [31:0] s13;

```

```
wire [31:0] s14;
```

```
wire [31:0] s15;
```

```
ripple_adder_32 a1 (.a(pp0), .b(pp1), .sum(s1));
```

```
ripple_adder_32 a2 (.a(s1), .b(pp2), .sum(s2));
```

```
ripple_adder_32 a3 (.a(s2), .b(pp3), .sum(s3));
```

```
ripple_adder_32 a4 (.a(s3), .b(pp4), .sum(s4));
```

```
ripple_adder_32 a5 (.a(s4), .b(pp5), .sum(s5));
```

```
ripple_adder_32 a6 (.a(s5), .b(pp6), .sum(s6));
```

```
ripple_adder_32 a7 (.a(s6), .b(pp7), .sum(s7));
```

```
ripple_adder_32 a8 (.a(s7), .b(pp8), .sum(s8));
```

```
ripple_adder_32 a9 (.a(s8), .b(pp9), .sum(s9));
```

```
ripple_adder_32 a10 (.a(s9), .b(pp10), .sum(s10));
```

```
ripple_adder_32 a11 (.a(s10), .b(pp11), .sum(s11));
```

```
ripple_adder_32 a12 (.a(s11), .b(pp12), .sum(s12));
```

```
ripple_adder_32 a13 (.a(s12), .b(pp13), .sum(s13));
```

```
ripple_adder_32 a14 (.a(s13), .b(pp14), .sum(s14));
```

```
ripple_adder_32 a15 (.a(s14), .b(pp15), .sum(s15));
```

```
assign product = s15;
```

```
endmodule
```

Prompt Strategy: Role Prompting

Prompt:

You are a Senior FPGA Design Engineer specializing in arithmetic accelerators and DSP architectures for Xilinx Artix-7 devices.

Generate production-quality synthesizable Verilog for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth algorithm.
- Structural RTL style only.
- Explicit hierarchical modules.
- Booth encoder modules.
- Partial-product generation modules.
- Structural adder network.
- No behavioral RTL.
- No multiplication operator (*).
- Optimized for Xilinx Artix-7 xc7a100tcsg324-2.
- Output code only.

Code:

```
module full_adder (  
    input wire a,  
    input wire b,  
    input wire cin,  
    output wire sum,  
    output wire cout  
);  
    assign sum = a ^ b ^ cin;  
    assign cout = (a & b) | (a & cin) | (b & cin);  
endmodule  
  
module ripple_adder_32 (  
    input wire [31:0] a,
```

```

input wire [31:0] b,

output wire [31:0] sum

);

wire [32:0] c;

assign c[0] = 1'b0;

genvar i;

generate

    for (i = 0; i < 32; i = i + 1) begin : GEN_FA

        full_adder fa (

            .a  (a[i]),

            .b  (b[i]),

            .cin (c[i]),

            .sum (sum[i]),

            .cout(c[i+1])

        );

    end

endgenerate

endmodule

```

```

module booth_encoder (

    input wire y_i,

    input wire y_im1,

    output wire add,

    output wire sub

```

```

);

    assign add = y_i & ~y_im1; // 01

    assign sub = ~y_i & y_im1; // 10

endmodule

```

```

module booth_partial_product (

    input wire [15:0] multiplicand,

    input wire      y_i,

    input wire      y_im1,

    input wire [4:0] shift,

    output wire [31:0] pp

);

```

```

    wire add_sel;

```

```

    wire sub_sel;

```

```

    wire [31:0] mcand_ext;

```

```

    wire [31:0] mcand_shifted;

```

```

    wire [31:0] mcand_inverted;

```

```

    wire [31:0] add_term;

```

```

    wire [31:0] sub_term;

```

```

    wire [31:0] sub_pp;

```

```

    booth_encoder enc (

```

```

        .y_i (y_i),

```

```

        .y_im1 (y_im1),

```

```

        .add (add_sel),

```



```

        .sub (sub_sel)

    );

    assign mcand_ext    = {{16{multiplicand[15]}}, multiplicand};
    assign mcand_shifted = mcand_ext << shift;
    assign mcand_inverted = ~mcand_shifted;

    assign add_term = add_sel ? mcand_shifted : 32'b0;
    assign sub_term = sub_sel ? mcand_inverted : 32'b0;

    ripple_adder_32 sub_add (
        .a (sub_term),
        .b ({31'b0, sub_sel}),
        .sum (sub_pp)
    );

    assign pp = add_term | sub_pp;
endmodule

module booth_multiplier_16x16_signed (
    input wire [15:0] a,
    input wire [15:0] b,
    output wire [31:0] product
);

    wire [31:0] pp0;
    wire [31:0] pp1;

```

```
wire [31:0] pp2;  
wire [31:0] pp3;  
wire [31:0] pp4;  
wire [31:0] pp5;  
wire [31:0] pp6;  
wire [31:0] pp7;  
wire [31:0] pp8;  
wire [31:0] pp9;  
wire [31:0] pp10;  
wire [31:0] pp11;  
wire [31:0] pp12;  
wire [31:0] pp13;  
wire [31:0] pp14;  
wire [31:0] pp15;
```

```
booth_partial_product pp_gen0 (  
    .multiplicand(a),  
    .y_i      (b[0]),  
    .y_im1    (1'b0),  
    .shift    (5'd0),  
    .pp       (pp0)  
);
```

```
booth_partial_product pp_gen1 (  
    .multiplicand(a),  
    .y_i      (b[1]),
```

```
.y_im1    (b[0]),  
.shift    (5'd1),  
.pp       (pp1)  
);
```

```
booth_partial_product pp_gen2 (  
    .multiplicand(a),  
    .y_i        (b[2]),  
    .y_im1      (b[1]),  
    .shift      (5'd2),  
    .pp         (pp2)  
);
```

```
booth_partial_product pp_gen3 (  
    .multiplicand(a),  
    .y_i        (b[3]),  
    .y_im1      (b[2]),  
    .shift      (5'd3),  
    .pp         (pp3)  
);
```

```
booth_partial_product pp_gen4 (  
    .multiplicand(a),  
    .y_i        (b[4]),  
    .y_im1      (b[3]),  
    .shift      (5'd4),
```

```
.pp      (pp4)

);
```

```
booth_partial_product pp_gen5 (

    .multiplicand(a),

    .y_i      (b[5]),

    .y_im1    (b[4]),

    .shift    (5'd5),

    .pp      (pp5)

);
```

```
booth_partial_product pp_gen6 (

    .multiplicand(a),

    .y_i      (b[6]),

    .y_im1    (b[5]),

    .shift    (5'd6),

    .pp      (pp6)

);
```

```
booth_partial_product pp_gen7 (

    .multiplicand(a),

    .y_i      (b[7]),

    .y_im1    (b[6]),

    .shift    (5'd7),

    .pp      (pp7)

);
```

```
booth_partial_product pp_gen8 (  
    .multiplicand(a),  
    .y_i      (b[8]),  
    .y_im1    (b[7]),  
    .shift    (5'd8),  
    .pp       (pp8)  
);
```

```
booth_partial_product pp_gen9 (  
    .multiplicand(a),  
    .y_i      (b[9]),  
    .y_im1    (b[8]),  
    .shift    (5'd9),  
    .pp       (pp9)  
);
```

```
booth_partial_product pp_gen10 (  
    .multiplicand(a),  
    .y_i      (b[10]),  
    .y_im1    (b[9]),  
    .shift    (5'd10),  
    .pp       (pp10)  
);
```

```
booth_partial_product pp_gen11 (  
    .multiplicand(a),  
    .y_i      (b[11]),  
    .y_im1    (b[10]),  
    .shift    (5'd11),  
    .pp       (pp11)  
);
```

```
.multiplicand(a),  
.y_i      (b[11]),  
.y_im1    (b[10]),  
.shift    (5'd11),  
.pp       (pp11)  
);
```

```
booth_partial_product pp_gen12 (  
.multiplicand(a),  
.y_i      (b[12]),  
.y_im1    (b[11]),  
.shift    (5'd12),  
.pp       (pp12)  
);
```

```
booth_partial_product pp_gen13 (  
.multiplicand(a),  
.y_i      (b[13]),  
.y_im1    (b[12]),  
.shift    (5'd13),  
.pp       (pp13)  
);
```

```
booth_partial_product pp_gen14 (  
.multiplicand(a),  
.y_i      (b[14]),
```

```
.y_im1    (b[13]),  
.shift    (5'd14),  
.pp       (pp14)  
);
```

```
booth_partial_product pp_gen15 (  
    .multiplicand(a),  
    .y_i         (b[15]),  
    .y_im1       (b[14]),  
    .shift       (5'd15),  
    .pp          (pp15)  
);
```

```
wire [31:0] s1;  
wire [31:0] s2;  
wire [31:0] s3;  
wire [31:0] s4;  
wire [31:0] s5;  
wire [31:0] s6;  
wire [31:0] s7;  
wire [31:0] s8;  
wire [31:0] s9;  
wire [31:0] s10;  
wire [31:0] s11;  
wire [31:0] s12;  
wire [31:0] s13;
```

```
wire [31:0] s14;
```

```
wire [31:0] s15;
```

```
ripple_adder_32 add1 (.a(pp0), .b(pp1), .sum(s1));
```

```
ripple_adder_32 add2 (.a(s1), .b(pp2), .sum(s2));
```

```
ripple_adder_32 add3 (.a(s2), .b(pp3), .sum(s3));
```

```
ripple_adder_32 add4 (.a(s3), .b(pp4), .sum(s4));
```

```
ripple_adder_32 add5 (.a(s4), .b(pp5), .sum(s5));
```

```
ripple_adder_32 add6 (.a(s5), .b(pp6), .sum(s6));
```

```
ripple_adder_32 add7 (.a(s6), .b(pp7), .sum(s7));
```

```
ripple_adder_32 add8 (.a(s7), .b(pp8), .sum(s8));
```

```
ripple_adder_32 add9 (.a(s8), .b(pp9), .sum(s9));
```

```
ripple_adder_32 add10 (.a(s9), .b(pp10), .sum(s10));
```

```
ripple_adder_32 add11 (.a(s10), .b(pp11), .sum(s11));
```

```
ripple_adder_32 add12 (.a(s11), .b(pp12), .sum(s12));
```

```
ripple_adder_32 add13 (.a(s12), .b(pp13), .sum(s13));
```

```
ripple_adder_32 add14 (.a(s13), .b(pp14), .sum(s14));
```

```
ripple_adder_32 add15 (.a(s14), .b(pp15), .sum(s15));
```

```
assign product = s15;
```

```
endmodule
```

Prompt Strategy: Instruction Format

Prompt:

TASK:

Generate a 16-bit signed Booth Multiplier.

ALGORITHM:

Radix-2 Booth Multiplication

LANGUAGE:

Verilog

RTL STYLE:

Structural

TARGET FPGA:

Xilinx Artix-7 xc7a100tcsg324-2

INPUTS:

input signed [15:0] multiplicand

input signed [15:0] multiplier

OUTPUT:

output signed [31:0] product

RULES:

- Use module instantiations.
- Use explicit hierarchy.
- Implement Booth encoding.

- Implement partial-product generation.
- Use structural adders.
- Do not use behavioral code.
- Do not use always blocks.
- Do not use multiplication operator (*).

OUTPUT FORMAT:

Verilog code only.

Code:

```
module full_adder(
    input a,
    input b,
    input cin,
    output sum,
    output cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (a & cin) | (b & cin);
endmodule
```

```
module ripple_adder_32(
    input [31:0] a,
    input [31:0] b,
    output [31:0] sum
```

```

);

wire [32:0] c;

assign c[0] = 1'b0;

genvar i;
generate
    for(i=0;i<32;i=i+1)
        begin: FA_CHAIN
            full_adder fa(
                .a(a[i]),
                .b(b[i]),
                .cin(c[i]),
                .sum(sum[i]),
                .cout(c[i+1])
            );
        end
    endgenerate
endmodule

```

```

module booth_encoder(
    input y_i,
    input y_im1,
    output add,
    output sub
);

```

```
assign add = y_i & ~y_im1;  
assign sub = ~y_i & y_im1;  
endmodule
```

```
module booth_partial_product(  
    input [15:0] multiplicand,  
    input      add,  
    input      sub,  
    input [4:0] shift,  
    output [31:0] pp  
);
```

```
wire [15:0] pos_data;  
wire [15:0] neg_data;
```

```
assign pos_data = multiplicand;  
assign neg_data = ~multiplicand;
```

```
wire [15:0] selected_data;
```

```
assign selected_data =  
    ({16{add}} & pos_data) |  
    ({16{sub}} & neg_data);
```

```
wire correction;
```

```
assign correction = sub;
```

```
wire [31:0] shifted_data;
```

```
wire [31:0] shifted_corr;
```

```
assign shifted_data = {{16{selected_data[15]}},selected_data} << shift;
```

```
assign shifted_corr = {{31{1'b0}},correction} << shift;
```

```
ripple_adder_32 corr_add(
```

```
    .a(shifted_data),
```

```
    .b(shifted_corr),
```

```
    .sum(pp)
```

```
);
```

```
endmodule
```

```
module booth_multiplier_16x16(
```

```
    input signed [15:0] multiplicand,
```

```
    input signed [15:0] multiplier,
```

```
    output signed [31:0] product
```

```
);
```

```
wire y_m1;
```

```
assign y_m1 = 1'b0;
```

```
/*-----
```

Booth Encoding

```
-----*/
```

```
wire add0, sub0;
```

```
wire add1, sub1;
```

```
wire add2, sub2;
```

```
wire add3, sub3;
```

```
wire add4, sub4;
```

```
wire add5, sub5;
```

```
wire add6, sub6;
```

```
wire add7, sub7;
```

```
wire add8, sub8;
```

```
wire add9, sub9;
```

```
wire add10, sub10;
```

```
wire add11, sub11;
```

```
wire add12, sub12;
```

```
wire add13, sub13;
```

```
wire add14, sub14;
```

```
wire add15, sub15;
```

```
booth_encoder be0 (.y_i(multiplier[0]), .y_im1(y_m1), .add(add0), .sub(sub0));
```

```
booth_encoder be1 (.y_i(multiplier[1]), .y_im1(multiplier[0]), .add(add1), .sub(sub1));
```

```
booth_encoder be2 (.y_i(multiplier[2]), .y_im1(multiplier[1]), .add(add2), .sub(sub2));
```

```
booth_encoder be3 (.y_i(multiplier[3]), .y_im1(multiplier[2]), .add(add3), .sub(sub3));
```

```
booth_encoder be4 (.y_i(multiplier[4]), .y_im1(multiplier[3]), .add(add4), .sub(sub4));
```

```

booth_encoder be5 (.y_i(multiplier[5]), .y_im1(multiplier[4]), .add(add5), .sub(sub5));
booth_encoder be6 (.y_i(multiplier[6]), .y_im1(multiplier[5]), .add(add6), .sub(sub6));
booth_encoder be7 (.y_i(multiplier[7]), .y_im1(multiplier[6]), .add(add7), .sub(sub7));
booth_encoder be8 (.y_i(multiplier[8]), .y_im1(multiplier[7]), .add(add8), .sub(sub8));
booth_encoder be9 (.y_i(multiplier[9]), .y_im1(multiplier[8]), .add(add9), .sub(sub9));
booth_encoder be10 (.y_i(multiplier[10]), .y_im1(multiplier[9]), .add(add10), .sub(sub10));
booth_encoder be11 (.y_i(multiplier[11]), .y_im1(multiplier[10]), .add(add11), .sub(sub11));
booth_encoder be12 (.y_i(multiplier[12]), .y_im1(multiplier[11]), .add(add12), .sub(sub12));
booth_encoder be13 (.y_i(multiplier[13]), .y_im1(multiplier[12]), .add(add13), .sub(sub13));
booth_encoder be14 (.y_i(multiplier[14]), .y_im1(multiplier[13]), .add(add14), .sub(sub14));
booth_encoder be15 (.y_i(multiplier[15]), .y_im1(multiplier[14]), .add(add15), .sub(sub15));

```

```

/*-----

```

Partial Products

```

-----*/

```

```

wire [31:0] pp0;

```

```

wire [31:0] pp1;

```

```

wire [31:0] pp2;

```

```

wire [31:0] pp3;

```

```

wire [31:0] pp4;

```

```

wire [31:0] pp5;

```

```

wire [31:0] pp6;

```

```

wire [31:0] pp7;

```

```

wire [31:0] pp8;

```

```

wire [31:0] pp9;

```

```
wire [31:0] pp10;  
  
wire [31:0] pp11;  
  
wire [31:0] pp12;  
  
wire [31:0] pp13;  
  
wire [31:0] pp14;  
  
wire [31:0] pp15;
```

```
booth_partial_product ppg0 (.multiplicand(multiplicand), .add(add0), .sub(sub0),  
.shift(5'd0), .pp(pp0));  
  
booth_partial_product ppg1 (.multiplicand(multiplicand), .add(add1), .sub(sub1),  
.shift(5'd1), .pp(pp1));  
  
booth_partial_product ppg2 (.multiplicand(multiplicand), .add(add2), .sub(sub2),  
.shift(5'd2), .pp(pp2));  
  
booth_partial_product ppg3 (.multiplicand(multiplicand), .add(add3), .sub(sub3),  
.shift(5'd3), .pp(pp3));  
  
booth_partial_product ppg4 (.multiplicand(multiplicand), .add(add4), .sub(sub4),  
.shift(5'd4), .pp(pp4));  
  
booth_partial_product ppg5 (.multiplicand(multiplicand), .add(add5), .sub(sub5),  
.shift(5'd5), .pp(pp5));  
  
booth_partial_product ppg6 (.multiplicand(multiplicand), .add(add6), .sub(sub6),  
.shift(5'd6), .pp(pp6));  
  
booth_partial_product ppg7 (.multiplicand(multiplicand), .add(add7), .sub(sub7),  
.shift(5'd7), .pp(pp7));  
  
booth_partial_product ppg8 (.multiplicand(multiplicand), .add(add8), .sub(sub8),  
.shift(5'd8), .pp(pp8));  
  
booth_partial_product ppg9 (.multiplicand(multiplicand), .add(add9), .sub(sub9),  
.shift(5'd9), .pp(pp9));  
  
booth_partial_product ppg10 (.multiplicand(multiplicand), .add(add10), .sub(sub10),  
.shift(5'd10), .pp(pp10));  
  
booth_partial_product ppg11 (.multiplicand(multiplicand), .add(add11), .sub(sub11),  
.shift(5'd11), .pp(pp11));
```



```
booth_partial_product ppg12 (.multiplicand(multiplicand), .add(add12), .sub(sub12),  
.shift(5'd12), .pp(pp12));
```

```
booth_partial_product ppg13 (.multiplicand(multiplicand), .add(add13), .sub(sub13),  
.shift(5'd13), .pp(pp13));
```

```
booth_partial_product ppg14 (.multiplicand(multiplicand), .add(add14), .sub(sub14),  
.shift(5'd14), .pp(pp14));
```

```
booth_partial_product ppg15 (.multiplicand(multiplicand), .add(add15), .sub(sub15),  
.shift(5'd15), .pp(pp15));
```

```
/*-----
```

Adder Tree

```
-----*/
```

```
wire [31:0] s01;
```

```
wire [31:0] s23;
```

```
wire [31:0] s45;
```

```
wire [31:0] s67;
```

```
wire [31:0] s89;
```

```
wire [31:0] s1011;
```

```
wire [31:0] s1213;
```

```
wire [31:0] s1415;
```

```
ripple_adder_32 a01 (.a(pp0), .b(pp1), .sum(s01));
```

```
ripple_adder_32 a23 (.a(pp2), .b(pp3), .sum(s23));
```

```
ripple_adder_32 a45 (.a(pp4), .b(pp5), .sum(s45));
```

```
ripple_adder_32 a67 (.a(pp6), .b(pp7), .sum(s67));
```

```
ripple_adder_32 a89 (.a(pp8), .b(pp9), .sum(s89));
```

```
ripple_adder_32 a1011 (.a(pp10), .b(pp11), .sum(s1011));
```

```
ripple_adder_32 a1213 (.a(pp12), .b(pp13), .sum(s1213));
```

```
ripple_adder_32 a1415 (.a(pp14), .b(pp15), .sum(s1415));
```

```
wire [31:0] s0_3;
```

```
wire [31:0] s4_7;
```

```
wire [31:0] s8_11;
```

```
wire [31:0] s12_15;
```

```
ripple_adder_32 a0_3 (.a(s01), .b(s23), .sum(s0_3));
```

```
ripple_adder_32 a4_7 (.a(s45), .b(s67), .sum(s4_7));
```

```
ripple_adder_32 a8_11 (.a(s89), .b(s1011), .sum(s8_11));
```

```
ripple_adder_32 a12_15 (.a(s1213), .b(s1415), .sum(s12_15));
```

```
wire [31:0] s0_7;
```

```
wire [31:0] s8_15;
```

```
ripple_adder_32 a0_7 (.a(s0_3), .b(s4_7), .sum(s0_7));
```

```
ripple_adder_32 a8_15 (.a(s8_11), .b(s12_15), .sum(s8_15));
```

```
ripple_adder_32 final_add(
```

```
    .a(s0_7),
```

```
    .b(s8_15),
```

```
    .sum(product)
```

```
);
```

```
endmodule
```

Prompt Strategy: Negative Prompting

Prompt:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth multiplication.
- Structural RTL only.

Do NOT:

- Use the multiplication operator (*).
- Use always blocks.
- Use always_comb.
- Use always_ff.
- Use behavioral arithmetic.
- Use high-level multiplication statements.
- Use behavioral datapaths.
- Use behavioral Booth implementations.

Must:

- Use Booth encoding.
- Use structural hierarchy.
- Use module instantiations.
- Use partial-product generation logic.

Target FPGA:

xc7a100tcsg324-2

Output code only.

Code:

```
module half_adder(  
    input a,  
    input b,  
    output sum,  
    output cout  
);  
assign sum = a ^ b;  
assign cout = a & b;  
endmodule
```

```
module full_adder(  
    input a,  
    input b,  
    input cin,  
    output sum,  
    output cout  
);  
wire s1;  
wire c1;
```

```
wire c2;
```

```
half_adder ha0(
```

```
    .a(a),
```

```
    .b(b),
```

```
    .sum(s1),
```

```
    .cout(c1)
```

```
);
```

```
half_adder ha1(
```

```
    .a(s1),
```

```
    .b(cin),
```

```
    .sum(sum),
```

```
    .cout(c2)
```

```
);
```

```
assign cout = c1 | c2;
```

```
endmodule
```

```
module ripple_adder_32(
```

```
    input [31:0] a,
```

```
    input [31:0] b,
```

```
    output [31:0] sum
```

```
);
```

```
wire [32:0] c;
```

```

assign c[0] = 1'b0;

genvar i;
generate
    for(i=0;i<32;i=i+1)
        begin : FA_CHAIN
            full_adder fa(
                .a(a[i]),
                .b(b[i]),
                .cin(c[i]),
                .sum(sum[i]),
                .cout(c[i+1])
            );
        end
    endgenerate

endmodule

```

```

module booth_encoder(
    input y_i,
    input y_im1,
    output add,
    output sub
);

```

```
assign add = (~y_i) & y_im1;
```

```
assign sub = y_i & (~y_im1);
```

```
endmodule
```

```
module booth_partial_product(
```

```
    input signed [15:0] multiplicand,
```

```
    input      add,
```

```
    input      sub,
```

```
    input  [4:0] shift,
```

```
    output [31:0] pp
```

```
);
```

```
wire [31:0] m_ext;
```

```
wire [31:0] m_shift;
```

```
wire [31:0] m_neg;
```

```
wire [31:0] add_term;
```

```
wire [31:0] sub_term;
```

```
assign m_ext = {{16{multiplicand[15]}}, multiplicand};
```

```
assign m_shift = m_ext <<< shift;
```

```
assign m_neg = ~m_shift + 32'd1;
```

```
assign add_term = {32{add}} & m_shift;
```

```
assign sub_term = {32{sub}} & m_neg;
```

```
assign pp = add_term | sub_term;
```

```
endmodule
```

```
module booth_multiplier_16(  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output signed [31:0] product  
);
```

```
wire [16:0] y;
```

```
assign y[0] = 1'b0;
```

```
assign y[16:1] = multiplier;
```

```
wire add0, sub0;
```

```
wire add1, sub1;
```

```
wire add2, sub2;
```

```
wire add3, sub3;
```

```
wire add4, sub4;
```

```
wire add5, sub5;
```

```
wire add6, sub6;
```

```
wire add7, sub7;
```

```
wire add8, sub8;
```

```
wire add9, sub9;
```

```
wire add10, sub10;
```


wire add11, sub11;

wire add12, sub12;

wire add13, sub13;

wire add14, sub14;

wire add15, sub15;

booth_encoder be0 (.y_i(y[1]), .y_im1(y[0]), .add(add0), .sub(sub0));

booth_encoder be1 (.y_i(y[2]), .y_im1(y[1]), .add(add1), .sub(sub1));

booth_encoder be2 (.y_i(y[3]), .y_im1(y[2]), .add(add2), .sub(sub2));

booth_encoder be3 (.y_i(y[4]), .y_im1(y[3]), .add(add3), .sub(sub3));

booth_encoder be4 (.y_i(y[5]), .y_im1(y[4]), .add(add4), .sub(sub4));

booth_encoder be5 (.y_i(y[6]), .y_im1(y[5]), .add(add5), .sub(sub5));

booth_encoder be6 (.y_i(y[7]), .y_im1(y[6]), .add(add6), .sub(sub6));

booth_encoder be7 (.y_i(y[8]), .y_im1(y[7]), .add(add7), .sub(sub7));

booth_encoder be8 (.y_i(y[9]), .y_im1(y[8]), .add(add8), .sub(sub8));

booth_encoder be9 (.y_i(y[10]), .y_im1(y[9]), .add(add9), .sub(sub9));

booth_encoder be10 (.y_i(y[11]), .y_im1(y[10]), .add(add10), .sub(sub10));

booth_encoder be11 (.y_i(y[12]), .y_im1(y[11]), .add(add11), .sub(sub11));

booth_encoder be12 (.y_i(y[13]), .y_im1(y[12]), .add(add12), .sub(sub12));

booth_encoder be13 (.y_i(y[14]), .y_im1(y[13]), .add(add13), .sub(sub13));

booth_encoder be14 (.y_i(y[15]), .y_im1(y[14]), .add(add14), .sub(sub14));

booth_encoder be15 (.y_i(y[16]), .y_im1(y[15]), .add(add15), .sub(sub15));

wire [31:0] pp0;

wire [31:0] pp1;

wire [31:0] pp2;

wire [31:0] pp3;

wire [31:0] pp4;

wire [31:0] pp5;

wire [31:0] pp6;

wire [31:0] pp7;

wire [31:0] pp8;

wire [31:0] pp9;

wire [31:0] pp10;

wire [31:0] pp11;

wire [31:0] pp12;

wire [31:0] pp13;

wire [31:0] pp14;

wire [31:0] pp15;

booth_partial_product ppg0 (.multiplicand(multiplicand), .add(add0), .sub(sub0),
.shift(5'd0), .pp(pp0));

booth_partial_product ppg1 (.multiplicand(multiplicand), .add(add1), .sub(sub1),
.shift(5'd1), .pp(pp1));

booth_partial_product ppg2 (.multiplicand(multiplicand), .add(add2), .sub(sub2),
.shift(5'd2), .pp(pp2));

booth_partial_product ppg3 (.multiplicand(multiplicand), .add(add3), .sub(sub3),
.shift(5'd3), .pp(pp3));

booth_partial_product ppg4 (.multiplicand(multiplicand), .add(add4), .sub(sub4),
.shift(5'd4), .pp(pp4));

booth_partial_product ppg5 (.multiplicand(multiplicand), .add(add5), .sub(sub5),
.shift(5'd5), .pp(pp5));

booth_partial_product ppg6 (.multiplicand(multiplicand), .add(add6), .sub(sub6),
.shift(5'd6), .pp(pp6));

booth_partial_product ppg7 (.multiplicand(multiplicand), .add(add7), .sub(sub7),
.shift(5'd7), .pp(pp7));

```
booth_partial_product ppg8 (.multiplicand(multiplicand), .add(add8), .sub(sub8),  
.shift(5'd8), .pp(pp8));  
  
booth_partial_product ppg9 (.multiplicand(multiplicand), .add(add9), .sub(sub9),  
.shift(5'd9), .pp(pp9));  
  
booth_partial_product ppg10 (.multiplicand(multiplicand), .add(add10), .sub(sub10),  
.shift(5'd10), .pp(pp10));  
  
booth_partial_product ppg11 (.multiplicand(multiplicand), .add(add11), .sub(sub11),  
.shift(5'd11), .pp(pp11));  
  
booth_partial_product ppg12 (.multiplicand(multiplicand), .add(add12), .sub(sub12),  
.shift(5'd12), .pp(pp12));  
  
booth_partial_product ppg13 (.multiplicand(multiplicand), .add(add13), .sub(sub13),  
.shift(5'd13), .pp(pp13));  
  
booth_partial_product ppg14 (.multiplicand(multiplicand), .add(add14), .sub(sub14),  
.shift(5'd14), .pp(pp14));  
  
booth_partial_product ppg15 (.multiplicand(multiplicand), .add(add15), .sub(sub15),  
.shift(5'd15), .pp(pp15));
```

```
wire [31:0] s01;
```

```
wire [31:0] s23;
```

```
wire [31:0] s45;
```

```
wire [31:0] s67;
```

```
wire [31:0] s89;
```

```
wire [31:0] s1011;
```

```
wire [31:0] s1213;
```

```
wire [31:0] s1415;
```

```
wire [31:0] s0123;
```

```
wire [31:0] s4567;
```

```
wire [31:0] s891011;
```

```
wire [31:0] s12131415;
```

```
wire [31:0] s0_7;
```

```
wire [31:0] s8_15;
```

```
ripple_adder_32 a01 (.a(pp0), .b(pp1), .sum(s01));
```

```
ripple_adder_32 a23 (.a(pp2), .b(pp3), .sum(s23));
```

```
ripple_adder_32 a45 (.a(pp4), .b(pp5), .sum(s45));
```

```
ripple_adder_32 a67 (.a(pp6), .b(pp7), .sum(s67));
```

```
ripple_adder_32 a89 (.a(pp8), .b(pp9), .sum(s89));
```

```
ripple_adder_32 a1011 (.a(pp10), .b(pp11), .sum(s1011));
```

```
ripple_adder_32 a1213 (.a(pp12), .b(pp13), .sum(s1213));
```

```
ripple_adder_32 a1415 (.a(pp14), .b(pp15), .sum(s1415));
```

```
ripple_adder_32 a0123 (.a(s01), .b(s23), .sum(s0123));
```

```
ripple_adder_32 a4567 (.a(s45), .b(s67), .sum(s4567));
```

```
ripple_adder_32 a891011 (.a(s89), .b(s1011), .sum(s891011));
```

```
ripple_adder_32 a12131415 (.a(s1213), .b(s1415), .sum(s12131415));
```

```
ripple_adder_32 a0_7 (.a(s0123), .b(s4567), .sum(s0_7));
```

```
ripple_adder_32 a8_15 (.a(s891011), .b(s12131415), .sum(s8_15));
```

```
ripple_adder_32 afinal (
```

```
    .a(s0_7),
```

```
    .b(s8_15),
```

```
    .sum(product)
```

```
);
```

endmodule

Prompt Strategy: Constraints First

Prompt:

Design Constraints:

1. Structural RTL only.
2. Synthesizable on Xilinx Artix-7 xc7a100tcsg324-2.
3. No multiplication operator (*).
4. No always blocks.
5. Explicit hierarchical design.
6. Booth encoder must be structurally implemented.
7. Partial products must be explicitly generated.
8. Structural adder network required.
9. Signed 16-bit multiplication.
10. Output width must be 32 bits.

Task:

Generate a Radix-2 Booth Multiplier satisfying all constraints above.

Output Verilog code only.

Code:

```
//=====

// 16-bit Signed Radix-2 Booth Multiplier

// Structural RTL

// No *, no always blocks, explicit hierarchy

//=====

module full_adder (

    input a,

    input b,

    input cin,

    output sum,

    output cout

);

    assign sum = a ^ b ^ cin;

    assign cout = (a & b) | (a & cin) | (b & cin);

endmodule

//=====

// 32-bit Ripple Carry Adder

//=====

module ripple_adder_32 (

    input [31:0] a,

    input [31:0] b,

    output [31:0] sum

);
```

```

wire [32:0] c;

assign c[0] = 1'b0;

genvar i;
generate
    for(i=0;i<32;i=i+1) begin : FA_GEN
        full_adder fa (
            .a  (a[i]),
            .b  (b[i]),
            .cin (c[i]),
            .sum (sum[i]),
            .cout(c[i+1])
        );
    end
endgenerate
endmodule

//=====================================================
// Booth Encoder
//=====================================================

module booth_encoder (
    input  y_i,
    input  y_im1,
    output add,
    output sub

```

```
);

    assign add = (~y_i) & y_im1; // 01

    assign sub = y_i & (~y_im1); // 10

endmodule
```

```
//=====

// Booth Partial Product Generator

// Produces shifted 32-bit partial product

//=====
```

```
module booth_partial_product (
    input signed [15:0] multiplicand,
    input      add,
    input      sub,
    input  [4:0] shift,
    output [31:0] pp
);
```

```
    wire [31:0] m_ext;
    wire [31:0] m_neg;
    wire [31:0] selected;
```

```
    assign m_ext = {{16{multiplicand[15]}}, multiplicand};
```

```
    assign m_neg = ~m_ext + 32'd1;
```

```
    assign selected =
```



```
(({32{add}} & m_ext) |
```

```
(({32{sub}} & m_neg);
```

```
assign pp = selected << shift;
```

```
endmodule
```

```
//=====
```

```
// Top-Level 16-bit Signed Radix-2 Booth Multiplier
```

```
//=====
```

```
module booth_multiplier_16 (
```

```
    input signed [15:0] multiplicand,
```

```
    input signed [15:0] multiplier,
```

```
    output signed [31:0] product
```

```
);
```

```
//-----
```

```
// Booth Encoding Signals
```

```
//-----
```

```
wire add0, sub0;
```

```
wire add1, sub1;
```

```
wire add2, sub2;
```

```
wire add3, sub3;
```

```
wire add4, sub4;
```

```
wire add5, sub5;
```

```
wire add6, sub6;
```

```
wire add7, sub7;

wire add8, sub8;

wire add9, sub9;

wire add10, sub10;

wire add11, sub11;

wire add12, sub12;

wire add13, sub13;

wire add14, sub14;

wire add15, sub15;
```

```
booth_encoder be0 (.y_i(multiplier[0]), .y_im1(1'b0), .add(add0), .sub(sub0));
booth_encoder be1 (.y_i(multiplier[1]), .y_im1(multiplier[0]), .add(add1), .sub(sub1));
booth_encoder be2 (.y_i(multiplier[2]), .y_im1(multiplier[1]), .add(add2), .sub(sub2));
booth_encoder be3 (.y_i(multiplier[3]), .y_im1(multiplier[2]), .add(add3), .sub(sub3));
booth_encoder be4 (.y_i(multiplier[4]), .y_im1(multiplier[3]), .add(add4), .sub(sub4));
booth_encoder be5 (.y_i(multiplier[5]), .y_im1(multiplier[4]), .add(add5), .sub(sub5));
booth_encoder be6 (.y_i(multiplier[6]), .y_im1(multiplier[5]), .add(add6), .sub(sub6));
booth_encoder be7 (.y_i(multiplier[7]), .y_im1(multiplier[6]), .add(add7), .sub(sub7));
booth_encoder be8 (.y_i(multiplier[8]), .y_im1(multiplier[7]), .add(add8), .sub(sub8));
booth_encoder be9 (.y_i(multiplier[9]), .y_im1(multiplier[8]), .add(add9), .sub(sub9));
booth_encoder be10 (.y_i(multiplier[10]), .y_im1(multiplier[9]), .add(add10), .sub(sub10));
booth_encoder be11 (.y_i(multiplier[11]), .y_im1(multiplier[10]), .add(add11), .sub(sub11));
booth_encoder be12 (.y_i(multiplier[12]), .y_im1(multiplier[11]), .add(add12), .sub(sub12));
booth_encoder be13 (.y_i(multiplier[13]), .y_im1(multiplier[12]), .add(add13), .sub(sub13));
booth_encoder be14 (.y_i(multiplier[14]), .y_im1(multiplier[13]), .add(add14), .sub(sub14));
booth_encoder be15 (.y_i(multiplier[15]), .y_im1(multiplier[14]), .add(add15), .sub(sub15));
```

```

//-----

// Partial Products

//-----

wire [31:0] pp0;

wire [31:0] pp1;

wire [31:0] pp2;

wire [31:0] pp3;

wire [31:0] pp4;

wire [31:0] pp5;

wire [31:0] pp6;

wire [31:0] pp7;

wire [31:0] pp8;

wire [31:0] pp9;

wire [31:0] pp10;

wire [31:0] pp11;

wire [31:0] pp12;

wire [31:0] pp13;

wire [31:0] pp14;

wire [31:0] pp15;


booth_partial_product ppg0 (multiplicand, add0, sub0, 5'd0, pp0);
booth_partial_product ppg1 (multiplicand, add1, sub1, 5'd1, pp1);
booth_partial_product ppg2 (multiplicand, add2, sub2, 5'd2, pp2);
booth_partial_product ppg3 (multiplicand, add3, sub3, 5'd3, pp3);
booth_partial_product ppg4 (multiplicand, add4, sub4, 5'd4, pp4);

```

```
booth_partial_product ppg5 (multiplicand, add5, sub5, 5'd5, pp5);
booth_partial_product ppg6 (multiplicand, add6, sub6, 5'd6, pp6);
booth_partial_product ppg7 (multiplicand, add7, sub7, 5'd7, pp7);
booth_partial_product ppg8 (multiplicand, add8, sub8, 5'd8, pp8);
booth_partial_product ppg9 (multiplicand, add9, sub9, 5'd9, pp9);
booth_partial_product ppg10 (multiplicand, add10, sub10, 5'd10, pp10);
booth_partial_product ppg11 (multiplicand, add11, sub11, 5'd11, pp11);
booth_partial_product ppg12 (multiplicand, add12, sub12, 5'd12, pp12);
booth_partial_product ppg13 (multiplicand, add13, sub13, 5'd13, pp13);
booth_partial_product ppg14 (multiplicand, add14, sub14, 5'd14, pp14);
booth_partial_product ppg15 (multiplicand, add15, sub15, 5'd15, pp15);
```

```
//-----
```

```
// Structural Adder Tree
```

```
//-----
```

```
wire [31:0] s1;
wire [31:0] s2;
wire [31:0] s3;
wire [31:0] s4;
wire [31:0] s5;
wire [31:0] s6;
wire [31:0] s7;
wire [31:0] s8;
wire [31:0] s9;
wire [31:0] s10;
wire [31:0] s11;
```

```
wire [31:0] s12;
```

```
wire [31:0] s13;
```

```
wire [31:0] s14;
```

```
wire [31:0] s15;
```

```
ripple_adder_32 a1 (.a(pp0), .b(pp1), .sum(s1));
```

```
ripple_adder_32 a2 (.a(s1), .b(pp2), .sum(s2));
```

```
ripple_adder_32 a3 (.a(s2), .b(pp3), .sum(s3));
```

```
ripple_adder_32 a4 (.a(s3), .b(pp4), .sum(s4));
```

```
ripple_adder_32 a5 (.a(s4), .b(pp5), .sum(s5));
```

```
ripple_adder_32 a6 (.a(s5), .b(pp6), .sum(s6));
```

```
ripple_adder_32 a7 (.a(s6), .b(pp7), .sum(s7));
```

```
ripple_adder_32 a8 (.a(s7), .b(pp8), .sum(s8));
```

```
ripple_adder_32 a9 (.a(s8), .b(pp9), .sum(s9));
```

```
ripple_adder_32 a10 (.a(s9), .b(pp10), .sum(s10));
```

```
ripple_adder_32 a11 (.a(s10), .b(pp11), .sum(s11));
```

```
ripple_adder_32 a12 (.a(s11), .b(pp12), .sum(s12));
```

```
ripple_adder_32 a13 (.a(s12), .b(pp13), .sum(s13));
```

```
ripple_adder_32 a14 (.a(s13), .b(pp14), .sum(s14));
```

```
ripple_adder_32 a15 (.a(s14), .b(pp15), .sum(s15));
```

```
assign product = s15;
```

```
endmodule
```

Prompt Strategy: Self Planning

Prompt:

Generate a 16-bit signed Booth Multiplier.

First create an implementation plan.

The plan should identify:

- Booth encoding blocks
- Partial-product generation blocks
- Adder hierarchy
- Datapath organization
- Structural module hierarchy

After creating the plan, implement the design according to the plan.

Requirements:

- Radix-2 Booth multiplication.
- Structural RTL only.
- Synthesizable Verilog.
- No behavioral blocks.
- No multiplication operator (*).
- Target FPGA: xc7a100tcsg324-2.

Output complete code.

Code:

```
module full_adder(  
    input a,  
    input b,  
    input cin,  
    output sum,  
    output cout  
);  
assign sum = a ^ b ^ cin;  
assign cout = (a & b) | (a & cin) | (b & cin);  
endmodule
```

```
module ripple_adder_32(  
    input [31:0] a,  
    input [31:0] b,  
    output [31:0] sum  
);
```

```
    wire [32:0] c;
```

```
    assign c[0] = 1'b0;
```

```
    genvar i;
```

```
    generate
```

```
        for(i=0;i<32;i=i+1)
```

```
begin:FA_GEN
    full_adder fa(
        .a (a[i]),
        .b (b[i]),
        .cin (c[i]),
        .sum (sum[i]),
        .cout(c[i+1])
    );
end
endgenerate

endmodule
```

```
module booth_encoder(
    input yi,
    input yim1,
    output add,
    output sub
);

assign add = (~yi) & yim1;
assign sub = yi & (~yim1);

endmodule
```



```

module partial_product_generator(
    input [15:0] multiplicand,
    input      add,
    input      sub,
    input [4:0] shift,
    output [31:0] pp
);

wire [31:0] m_ext;
wire [31:0] m_shift;
wire [31:0] m_neg;

assign m_ext  = {{16{multiplicand[15]}}, multiplicand};
assign m_shift = m_ext << shift;

assign m_neg = ~m_shift + 32'd1;

assign pp =
    add ? m_shift :
    sub ? m_neg  :
    32'd0;

endmodule

```

```
module booth_multiplier_16(  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output signed [31:0] product  
);
```

```
wire [15:0] add_sel;  
wire [15:0] sub_sel;
```

```
booth_encoder be0 (  
    .yi(multiplier[0]),  
    .yim1(1'b0),  
    .add(add_sel[0]),  
    .sub(sub_sel[0])  
);
```

```
genvar k;
```

```
generate
```

```
for(k=1;k<16;k=k+1)
```

```
begin:ENCODERS
```

```
    booth_encoder be(  
        .yi(multiplier[k]),  
        .yim1(multiplier[k-1]),  
        .add(add_sel[k]),  
        .sub(sub_sel[k])
```

```
    );  
end  
endgenerate
```

```
wire [31:0] pp0;  
wire [31:0] pp1;  
wire [31:0] pp2;  
wire [31:0] pp3;  
wire [31:0] pp4;  
wire [31:0] pp5;  
wire [31:0] pp6;  
wire [31:0] pp7;  
wire [31:0] pp8;  
wire [31:0] pp9;  
wire [31:0] pp10;  
wire [31:0] pp11;  
wire [31:0] pp12;  
wire [31:0] pp13;  
wire [31:0] pp14;  
wire [31:0] pp15;
```

```
partial_product_generator ppg0 (  
    .multiplicand(multiplicand),  
    .add(add_sel[0]),  
    .sub(sub_sel[0]),
```

```
.shift(5'd0),  
  
.pp(pp0)  
);
```

```
partial_product_generator ppg1 (  
    .multiplicand(multiplicand),  
    .add(add_sel[1]),  
    .sub(sub_sel[1]),  
    .shift(5'd1),  
    .pp(pp1)  
);
```

```
partial_product_generator ppg2 (  
    .multiplicand(multiplicand),  
    .add(add_sel[2]),  
    .sub(sub_sel[2]),  
    .shift(5'd2),  
    .pp(pp2)  
);
```

```
partial_product_generator ppg3 (  
    .multiplicand(multiplicand),  
    .add(add_sel[3]),  
    .sub(sub_sel[3]),  
    .shift(5'd3),  
    .pp(pp3)
```

);

partial_product_generator ppg4 (

.multiplicand(multiplicand),

.add(add_sel[4]),

.sub(sub_sel[4]),

.shift(5'd4),

.pp(pp4)

);

partial_product_generator ppg5 (

.multiplicand(multiplicand),

.add(add_sel[5]),

.sub(sub_sel[5]),

.shift(5'd5),

.pp(pp5)

);

partial_product_generator ppg6 (

.multiplicand(multiplicand),

.add(add_sel[6]),

.sub(sub_sel[6]),

.shift(5'd6),

.pp(pp6)

);

```
partial_product_generator ppg7 (  
    .multiplicand(multiplicand),  
    .add(add_sel[7]),  
    .sub(sub_sel[7]),  
    .shift(5'd7),  
    .pp(pp7)  
);
```

```
partial_product_generator ppg8 (  
    .multiplicand(multiplicand),  
    .add(add_sel[8]),  
    .sub(sub_sel[8]),  
    .shift(5'd8),  
    .pp(pp8)  
);
```

```
partial_product_generator ppg9 (  
    .multiplicand(multiplicand),  
    .add(add_sel[9]),  
    .sub(sub_sel[9]),  
    .shift(5'd9),  
    .pp(pp9)  
);
```

```
partial_product_generator ppg10 (  
    .multiplicand(multiplicand),
```

```
.add(add_sel[10]),  
.sub(sub_sel[10]),  
.shift(5'd10),  
.pp(pp10)  
);
```

```
partial_product_generator ppg11 (  
    .multiplicand(multiplicand),  
    .add(add_sel[11]),  
    .sub(sub_sel[11]),  
    .shift(5'd11),  
    .pp(pp11)  
);
```

```
partial_product_generator ppg12 (  
    .multiplicand(multiplicand),  
    .add(add_sel[12]),  
    .sub(sub_sel[12]),  
    .shift(5'd12),  
    .pp(pp12)  
);
```

```
partial_product_generator ppg13 (  
    .multiplicand(multiplicand),  
    .add(add_sel[13]),  
    .sub(sub_sel[13]),
```

```
.shift(5'd13),  
  
.pp(pp13)  
);
```

```
partial_product_generator ppg14 (  
  
.multiplicand(multiplicand),  
  
.add(add_sel[14]),  
  
.sub(sub_sel[14]),  
  
.shift(5'd14),  
  
.pp(pp14)  
);
```

```
partial_product_generator ppg15 (  
  
.multiplicand(multiplicand),  
  
.add(add_sel[15]),  
  
.sub(sub_sel[15]),  
  
.shift(5'd15),  
  
.pp(pp15)  
);
```

```
wire [31:0] s1_0;  
  
wire [31:0] s1_1;  
  
wire [31:0] s1_2;  
  
wire [31:0] s1_3;  
  
wire [31:0] s1_4;
```



```
wire [31:0] s1_5;
```

```
wire [31:0] s1_6;
```

```
wire [31:0] s1_7;
```

```
ripple_adder_32 a10(.a(pp0), .b(pp1), .sum(s1_0));
```

```
ripple_adder_32 a11(.a(pp2), .b(pp3), .sum(s1_1));
```

```
ripple_adder_32 a12(.a(pp4), .b(pp5), .sum(s1_2));
```

```
ripple_adder_32 a13(.a(pp6), .b(pp7), .sum(s1_3));
```

```
ripple_adder_32 a14(.a(pp8), .b(pp9), .sum(s1_4));
```

```
ripple_adder_32 a15(.a(pp10), .b(pp11), .sum(s1_5));
```

```
ripple_adder_32 a16(.a(pp12), .b(pp13), .sum(s1_6));
```

```
ripple_adder_32 a17(.a(pp14), .b(pp15), .sum(s1_7));
```

```
wire [31:0] s2_0;
```

```
wire [31:0] s2_1;
```

```
wire [31:0] s2_2;
```

```
wire [31:0] s2_3;
```

```
ripple_adder_32 a20(.a(s1_0), .b(s1_1), .sum(s2_0));
```

```
ripple_adder_32 a21(.a(s1_2), .b(s1_3), .sum(s2_1));
```

```
ripple_adder_32 a22(.a(s1_4), .b(s1_5), .sum(s2_2));
```

```
ripple_adder_32 a23(.a(s1_6), .b(s1_7), .sum(s2_3));
```

```
wire [31:0] s3_0;
```

```
wire [31:0] s3_1;
```

```
ripple_adder_32 a30(.a(s2_0), .b(s2_1), .sum(s3_0));  
ripple_adder_32 a31(.a(s2_2), .b(s2_3), .sum(s3_1));  
  
wire [31:0] s4;  
  
ripple_adder_32 a40(.a(s3_0), .b(s3_1), .sum(s4));  
  
assign product = s4;  
  
endmodule
```

Prompt Strategy: Iterative Correction

Prompt:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

After generating the initial design:

1. Review the RTL for violations of Structural RTL style.
2. Remove any behavioral constructs.
3. Remove any multiplication operators.
4. Verify that Booth encoding is explicitly implemented.
5. Verify that module hierarchy is present.
6. Verify synthesizability.
7. Correct any identified issues.

Then provide the final corrected version.

Requirements:

- Radix-2 Booth multiplication.
- Structural RTL only.
- Target FPGA: xc7a100tcsg324-2.
- Output final code only.

Code:

```
module full_adder (  
    input wire a,  
    input wire b,  
    input wire cin,  
    output wire sum,  
    output wire cout  
);  
    assign sum = a ^ b ^ cin;  
    assign cout = (a & b) | (a & cin) | (b & cin);  
endmodule
```

```
module ripple_adder_32 (  
    input wire [31:0] a,  
    input wire [31:0] b,  
    output wire [31:0] sum  
);
```

```

wire [32:0] c;

assign c[0] = 1'b0;

genvar i;
generate
    for (i = 0; i < 32; i = i + 1) begin : FA_CHAIN
        full_adder fa (
            .a (a[i]),
            .b (b[i]),
            .cin (c[i]),
            .sum (sum[i]),
            .cout(c[i+1])
        );
    end
endgenerate
endmodule

module booth_encoder (
    input wire q_i,
    input wire q_im1,
    output wire add,
    output wire sub
);

assign add = q_i & ~q_im1; // 10 -> -M handled by sub
assign sub = ~q_i & q_im1; // 01 -> +M handled by add path

```

```
endmodule
```

```
module booth_partial_product (  
    input wire signed [15:0] multiplicand,  
    input wire          add,  
    input wire          sub,  
    input wire [4:0]     shift_amt,  
    output wire [31:0]    pp  
);  
  
    wire signed [31:0] mcand_ext;  
    wire signed [31:0] shifted_mcand;  
    wire signed [31:0] pos_pp;  
    wire signed [31:0] neg_pp;  
  
    assign mcand_ext  = {{16{multiplicand[15]}}, multiplicand};  
    assign shifted_mcand = mcand_ext <<< shift_amt;  
  
    assign pos_pp = add ? shifted_mcand : 32'd0;  
    assign neg_pp = sub ? (~shifted_mcand + 32'd1) : 32'd0;  
  
    assign pp = pos_pp | neg_pp;  
endmodule
```

```
module booth_multiplier_16 (  
    input wire signed [15:0] multiplicand,  
    input wire signed [15:0] multiplier,
```

```
output wire signed [31:0] product  
);
```

```
wire [15:0] add_sel;  
wire [15:0] sub_sel;
```

```
wire [31:0] pp0;  
wire [31:0] pp1;  
wire [31:0] pp2;  
wire [31:0] pp3;  
wire [31:0] pp4;  
wire [31:0] pp5;  
wire [31:0] pp6;  
wire [31:0] pp7;  
wire [31:0] pp8;  
wire [31:0] pp9;  
wire [31:0] pp10;  
wire [31:0] pp11;  
wire [31:0] pp12;  
wire [31:0] pp13;  
wire [31:0] pp14;  
wire [31:0] pp15;
```

```
booth_encoder be0 (.q_i(multiplier[0]), .q_im1(1'b0),  
.add(add_sel[0]),  
.sub(sub_sel[0]));
```

```
booth_encoder be1 (.q_i(multiplier[1]), .q_im1(multiplier[0]), .add(add_sel[1]),  
.sub(sub_sel[1]));
```

```

    booth_encoder be2 (.q_i(multiplier[2]), .q_im1(multiplier[1]), .add(add_sel[2]),
.sub(sub_sel[2]));

    booth_encoder be3 (.q_i(multiplier[3]), .q_im1(multiplier[2]), .add(add_sel[3]),
.sub(sub_sel[3]));

    booth_encoder be4 (.q_i(multiplier[4]), .q_im1(multiplier[3]), .add(add_sel[4]),
.sub(sub_sel[4]));

    booth_encoder be5 (.q_i(multiplier[5]), .q_im1(multiplier[4]), .add(add_sel[5]),
.sub(sub_sel[5]));

    booth_encoder be6 (.q_i(multiplier[6]), .q_im1(multiplier[5]), .add(add_sel[6]),
.sub(sub_sel[6]));

    booth_encoder be7 (.q_i(multiplier[7]), .q_im1(multiplier[6]), .add(add_sel[7]),
.sub(sub_sel[7]));

    booth_encoder be8 (.q_i(multiplier[8]), .q_im1(multiplier[7]), .add(add_sel[8]),
.sub(sub_sel[8]));

    booth_encoder be9 (.q_i(multiplier[9]), .q_im1(multiplier[8]), .add(add_sel[9]),
.sub(sub_sel[9]));

    booth_encoder be10 (.q_i(multiplier[10]), .q_im1(multiplier[9]), .add(add_sel[10]),
.sub(sub_sel[10]));

    booth_encoder be11 (.q_i(multiplier[11]), .q_im1(multiplier[10]), .add(add_sel[11]),
.sub(sub_sel[11]));

    booth_encoder be12 (.q_i(multiplier[12]), .q_im1(multiplier[11]), .add(add_sel[12]),
.sub(sub_sel[12]));

    booth_encoder be13 (.q_i(multiplier[13]), .q_im1(multiplier[12]), .add(add_sel[13]),
.sub(sub_sel[13]));

    booth_encoder be14 (.q_i(multiplier[14]), .q_im1(multiplier[13]), .add(add_sel[14]),
.sub(sub_sel[14]));

    booth_encoder be15 (.q_i(multiplier[15]), .q_im1(multiplier[14]), .add(add_sel[15]),
.sub(sub_sel[15]));


    booth_partial_product ppg0 (.multiplicand(multiplicand), .add(sub_sel[0]),
.sub(add_sel[0]), .shift_amt(5'd0), .pp(pp0));

    booth_partial_product ppg1 (.multiplicand(multiplicand), .add(sub_sel[1]),
.sub(add_sel[1]), .shift_amt(5'd1), .pp(pp1));

```

```

    booth_partial_product ppg2 (.multiplicand(multiplicand), .add(sub_sel[2]),
    .sub(add_sel[2]), .shift_amt(5'd2), .pp(pp2));

    booth_partial_product ppg3 (.multiplicand(multiplicand), .add(sub_sel[3]),
    .sub(add_sel[3]), .shift_amt(5'd3), .pp(pp3));

    booth_partial_product ppg4 (.multiplicand(multiplicand), .add(sub_sel[4]),
    .sub(add_sel[4]), .shift_amt(5'd4), .pp(pp4));

    booth_partial_product ppg5 (.multiplicand(multiplicand), .add(sub_sel[5]),
    .sub(add_sel[5]), .shift_amt(5'd5), .pp(pp5));

    booth_partial_product ppg6 (.multiplicand(multiplicand), .add(sub_sel[6]),
    .sub(add_sel[6]), .shift_amt(5'd6), .pp(pp6));

    booth_partial_product ppg7 (.multiplicand(multiplicand), .add(sub_sel[7]),
    .sub(add_sel[7]), .shift_amt(5'd7), .pp(pp7));

    booth_partial_product ppg8 (.multiplicand(multiplicand), .add(sub_sel[8]),
    .sub(add_sel[8]), .shift_amt(5'd8), .pp(pp8));

    booth_partial_product ppg9 (.multiplicand(multiplicand), .add(sub_sel[9]),
    .sub(add_sel[9]), .shift_amt(5'd9), .pp(pp9));

    booth_partial_product ppg10 (.multiplicand(multiplicand), .add(sub_sel[10]),
    .sub(add_sel[10]), .shift_amt(5'd10), .pp(pp10));

    booth_partial_product ppg11 (.multiplicand(multiplicand), .add(sub_sel[11]),
    .sub(add_sel[11]), .shift_amt(5'd11), .pp(pp11));

    booth_partial_product ppg12 (.multiplicand(multiplicand), .add(sub_sel[12]),
    .sub(add_sel[12]), .shift_amt(5'd12), .pp(pp12));

    booth_partial_product ppg13 (.multiplicand(multiplicand), .add(sub_sel[13]),
    .sub(add_sel[13]), .shift_amt(5'd13), .pp(pp13));

    booth_partial_product ppg14 (.multiplicand(multiplicand), .add(sub_sel[14]),
    .sub(add_sel[14]), .shift_amt(5'd14), .pp(pp14));

    booth_partial_product ppg15 (.multiplicand(multiplicand), .add(sub_sel[15]),
    .sub(add_sel[15]), .shift_amt(5'd15), .pp(pp15));

```

```
wire [31:0] s1;
```

```
wire [31:0] s2;
```

```
wire [31:0] s3;
```

```
wire [31:0] s4;
```



```
wire [31:0] s5;  
wire [31:0] s6;  
wire [31:0] s7;  
wire [31:0] s8;  
wire [31:0] s9;  
wire [31:0] s10;  
wire [31:0] s11;  
wire [31:0] s12;  
wire [31:0] s13;  
wire [31:0] s14;  
wire [31:0] s15;
```

```
ripple_adder_32 a1 (.a(pp0), .b(pp1), .sum(s1));  
ripple_adder_32 a2 (.a(s1), .b(pp2), .sum(s2));  
ripple_adder_32 a3 (.a(s2), .b(pp3), .sum(s3));  
ripple_adder_32 a4 (.a(s3), .b(pp4), .sum(s4));  
ripple_adder_32 a5 (.a(s4), .b(pp5), .sum(s5));  
ripple_adder_32 a6 (.a(s5), .b(pp6), .sum(s6));  
ripple_adder_32 a7 (.a(s6), .b(pp7), .sum(s7));  
ripple_adder_32 a8 (.a(s7), .b(pp8), .sum(s8));  
ripple_adder_32 a9 (.a(s8), .b(pp9), .sum(s9));  
ripple_adder_32 a10 (.a(s9), .b(pp10), .sum(s10));  
ripple_adder_32 a11 (.a(s10), .b(pp11), .sum(s11));  
ripple_adder_32 a12 (.a(s11), .b(pp12), .sum(s12));  
ripple_adder_32 a13 (.a(s12), .b(pp13), .sum(s13));  
ripple_adder_32 a14 (.a(s13), .b(pp14), .sum(s14));
```

```
ripple_adder_32 a15 (.a(s14), .b(pp15), .sum(s15));
```

```
assign product = s15;
```

```
endmodule
```

Prompt Strategy: Hybrid

Prompt:

You are a Senior FPGA Design Engineer.

TASK:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

Before coding:

1. Analyze the Booth algorithm.
2. Create an implementation plan.
3. Determine module hierarchy.
4. Determine partial-product generation strategy.
5. Determine adder architecture.

CONSTRAINTS:

- Structural RTL only.
- No multiplication operator (*).
- No behavioral blocks.
- No always, always_comb, or always_ff.

- Explicit module instantiations required.
- Implement Radix-2 Booth encoding.
- Target FPGA: Xilinx Artix-7 xc7a100tcsg324-2.

SELF-CHECK:

- Verify structural compliance.
- Verify Booth encoding implementation.
- Verify synthesizability.
- Verify correct port widths.

OUTPUT:

Verilog code only.

Code:

```
//
=====
=====

// 16-bit Signed Radix-2 Booth Multiplier

// Structural RTL

// Target: Xilinx Artix-7 xc7a100tcsg324-2

//

// Features:

// - Radix-2 Booth encoding

// - Explicit partial-product generation

// - Structural hierarchy

// - No multiplication operator
```

```
// - No always blocks
```

```
// - Synthesizable Verilog
```

```
//
```

```
=====
```

```
module half_adder (
```

```
    input a,
```

```
    input b,
```

```
    output sum,
```

```
    output cout
```

```
);
```

```
    assign sum = a ^ b;
```

```
    assign cout = a & b;
```

```
endmodule
```

```
module full_adder (
```

```
    input a,
```

```
    input b,
```

```
    input cin,
```

```
    output sum,
```

```
    output cout
```

```
);
```

```
    wire s1;
```

```
    wire c1;
```

```
    wire c2;
```

```

    assign s1  = a ^ b;

    assign sum = s1 ^ cin;

    assign c1  = a & b;

    assign c2  = s1 & cin;

    assign cout = c1 | c2;

endmodule


// -----
// Structural Ripple Carry Adder
// -----


module ripple_adder_32 (
    input  [31:0] a,
    input  [31:0] b,
    output [31:0] sum
);

    wire [32:0] c;

    assign c[0] = 1'b0;

    genvar i;
    generate
        for(i=0;i<32;i=i+1)
            begin : FA_GEN
                full_adder fa (
                    .a  (a[i]),

```

```

        .b (b[i]),

        .cin (c[i]),

        .sum (sum[i]),

        .cout(c[i+1])

    );

end

endgenerate

endmodule

```

```

// -----

// Booth Encoder

//

// Booth Table:

//

// y[i] y[i-1]

// 0  0 -> 0

// 0  1 -> +M

// 1  0 -> -M

// 1  1 -> 0

// -----

```

```

module booth_encoder (

    input y_i,

    input y_im1,

    output add,

    output sub

```

```
);

    assign add = (~y_i) & y_im1;

    assign sub = y_i & (~y_im1);

endmodule
```

```
// -----

// Booth Partial Product Generator

// Generates:

// 0

// +(M << shift)

// -(M << shift)

//

// Output width = 32 bits

// -----
```

```
module booth_partial_product #(

    parameter SHIFT = 0

)(

    input  [15:0] multiplicand,

    input      add,

    input      sub,

    output [31:0] pp

);

    wire [31:0] m_ext;

    wire [31:0] m_shift;
```

```

wire [31:0] neg_shift;

assign m_ext  = {{16{multiplicand[15]}}, multiplicand};

assign m_shift = m_ext << SHIFT;

assign neg_shift = ~m_shift + 32'd1;

assign pp =
    ({32{add}} & m_shift)
    | ({32{sub}} & neg_shift);

endmodule

// -----
// Top-Level 16-bit Signed Booth Multiplier
// -----

module booth_multiplier_16 (
    input  signed [15:0] multiplicand,
    input  signed [15:0] multiplier,
    output signed [31:0] product
);

// -----
// Booth control signals
// -----

```



```
wire add0, sub0;

wire add1, sub1;

wire add2, sub2;

wire add3, sub3;

wire add4, sub4;

wire add5, sub5;

wire add6, sub6;

wire add7, sub7;

wire add8, sub8;

wire add9, sub9;

wire add10, sub10;

wire add11, sub11;

wire add12, sub12;

wire add13, sub13;

wire add14, sub14;

wire add15, sub15;
```

```
booth_encoder be0 (
    .y_i  (multiplier[0]),
    .y_im1 (1'b0),
    .add  (add0),
    .sub  (sub0)
);
```

```
booth_encoder be1 (
```

```
.y_i (multiplier[1]),  
.y_im1 (multiplier[0]),  
.add (add1),  
.sub (sub1)  
);
```

```
booth_encoder be2 (  
.y_i (multiplier[2]),  
.y_im1 (multiplier[1]),  
.add (add2),  
.sub (sub2)  
);
```

```
booth_encoder be3 (  
.y_i (multiplier[3]),  
.y_im1 (multiplier[2]),  
.add (add3),  
.sub (sub3)  
);
```

```
booth_encoder be4 (  
.y_i (multiplier[4]),  
.y_im1 (multiplier[3]),  
.add (add4),  
.sub (sub4)  
);
```

```
booth_encoder be5 (  
    .y_i (multiplier[5]),  
    .y_im1 (multiplier[4]),  
    .add (add5),  
    .sub (sub5)  
);
```

```
booth_encoder be6 (  
    .y_i (multiplier[6]),  
    .y_im1 (multiplier[5]),  
    .add (add6),  
    .sub (sub6)  
);
```

```
booth_encoder be7 (  
    .y_i (multiplier[7]),  
    .y_im1 (multiplier[6]),  
    .add (add7),  
    .sub (sub7)  
);
```

```
booth_encoder be8 (  
    .y_i (multiplier[8]),  
    .y_im1 (multiplier[7]),  
    .add (add8),
```

```
.sub (sub8)

);

booth_encoder be9 (
    .y_i (multiplier[9]),
    .y_im1 (multiplier[8]),
    .add (add9),
    .sub (sub9)
);
```

```
booth_encoder be10 (
    .y_i (multiplier[10]),
    .y_im1 (multiplier[9]),
    .add (add10),
    .sub (sub10)
);
```

```
booth_encoder be11 (
    .y_i (multiplier[11]),
    .y_im1 (multiplier[10]),
    .add (add11),
    .sub (sub11)
);
```

```
booth_encoder be12 (
    .y_i (multiplier[12]),
```

```
.y_im1 (multiplier[11]),  
.add (add12),  
.sub (sub12)  
);
```

```
booth_encoder be13 (  
.y_i (multiplier[13]),  
.y_im1 (multiplier[12]),  
.add (add13),  
.sub (sub13)  
);
```

```
booth_encoder be14 (  
.y_i (multiplier[14]),  
.y_im1 (multiplier[13]),  
.add (add14),  
.sub (sub14)  
);
```

```
booth_encoder be15 (  
.y_i (multiplier[15]),  
.y_im1 (multiplier[14]),  
.add (add15),  
.sub (sub15)  
);
```

```
// -----
```

```
// Partial Products
```

```
// -----
```

```
wire [31:0] pp0;
```

```
wire [31:0] pp1;
```

```
wire [31:0] pp2;
```

```
wire [31:0] pp3;
```

```
wire [31:0] pp4;
```

```
wire [31:0] pp5;
```

```
wire [31:0] pp6;
```

```
wire [31:0] pp7;
```

```
wire [31:0] pp8;
```

```
wire [31:0] pp9;
```

```
wire [31:0] pp10;
```

```
wire [31:0] pp11;
```

```
wire [31:0] pp12;
```

```
wire [31:0] pp13;
```

```
wire [31:0] pp14;
```

```
wire [31:0] pp15;
```

```
    booth_partial_product #(0) p0 (.multiplicand(multiplicand), .add(add0), .sub(sub0),  
    .pp(pp0));
```

```
    booth_partial_product #(1) p1 (.multiplicand(multiplicand), .add(add1), .sub(sub1),  
    .pp(pp1));
```

```
    booth_partial_product #(2) p2 (.multiplicand(multiplicand), .add(add2), .sub(sub2),  
    .pp(pp2));
```

```

    booth_partial_product #(3) p3 (.multiplicand(multiplicand), .add(add3), .sub(sub3),
.pp(pp3));

    booth_partial_product #(4) p4 (.multiplicand(multiplicand), .add(add4), .sub(sub4),
.pp(pp4));

    booth_partial_product #(5) p5 (.multiplicand(multiplicand), .add(add5), .sub(sub5),
.pp(pp5));

    booth_partial_product #(6) p6 (.multiplicand(multiplicand), .add(add6), .sub(sub6),
.pp(pp6));

    booth_partial_product #(7) p7 (.multiplicand(multiplicand), .add(add7), .sub(sub7),
.pp(pp7));

    booth_partial_product #(8) p8 (.multiplicand(multiplicand), .add(add8), .sub(sub8),
.pp(pp8));

    booth_partial_product #(9) p9 (.multiplicand(multiplicand), .add(add9), .sub(sub9),
.pp(pp9));

    booth_partial_product #(10) p10 (.multiplicand(multiplicand), .add(add10), .sub(sub10),
.pp(pp10));

    booth_partial_product #(11) p11 (.multiplicand(multiplicand), .add(add11), .sub(sub11),
.pp(pp11));

    booth_partial_product #(12) p12 (.multiplicand(multiplicand), .add(add12), .sub(sub12),
.pp(pp12));

    booth_partial_product #(13) p13 (.multiplicand(multiplicand), .add(add13), .sub(sub13),
.pp(pp13));

    booth_partial_product #(14) p14 (.multiplicand(multiplicand), .add(add14), .sub(sub14),
.pp(pp14));

    booth_partial_product #(15) p15 (.multiplicand(multiplicand), .add(add15), .sub(sub15),
.pp(pp15));

```

```
// -----
```

```
// Structural Adder Tree
```

```
// -----
```

```
wire [31:0] s01;
```

```
wire [31:0] s23;  
  
wire [31:0] s45;  
  
wire [31:0] s67;  
  
wire [31:0] s89;  
  
wire [31:0] s1011;  
  
wire [31:0] s1213;  
  
wire [31:0] s1415;
```

```
ripple_adder_32 a01 (.a(pp0), .b(pp1), .sum(s01));  
ripple_adder_32 a23 (.a(pp2), .b(pp3), .sum(s23));  
ripple_adder_32 a45 (.a(pp4), .b(pp5), .sum(s45));  
ripple_adder_32 a67 (.a(pp6), .b(pp7), .sum(s67));  
ripple_adder_32 a89 (.a(pp8), .b(pp9), .sum(s89));  
ripple_adder_32 a1011 (.a(pp10), .b(pp11), .sum(s1011));  
ripple_adder_32 a1213 (.a(pp12), .b(pp13), .sum(s1213));  
ripple_adder_32 a1415 (.a(pp14), .b(pp15), .sum(s1415));
```

```
wire [31:0] s0123;  
  
wire [31:0] s4567;  
  
wire [31:0] s891011;  
  
wire [31:0] s12131415;
```

```
ripple_adder_32 a0123 (.a(s01), .b(s23), .sum(s0123));  
ripple_adder_32 a4567 (.a(s45), .b(s67), .sum(s4567));  
ripple_adder_32 a891011 (.a(s89), .b(s1011), .sum(s891011));  
ripple_adder_32 a12131415 (.a(s1213), .b(s1415), .sum(s12131415));
```



```
wire [31:0] s0_7;
```

```
wire [31:0] s8_15;
```

```
ripple_adder_32 a0_7 (.a(s0123), .b(s4567), .sum(s0_7));
```

```
ripple_adder_32 a8_15 (.a(s891011), .b(s12131415), .sum(s8_15));
```

```
ripple_adder_32 afinal (
```

```
    .a (s0_7),
```

```
    .b (s8_15),
```

```
    .sum (product)
```

```
);
```

```
endmodule
```